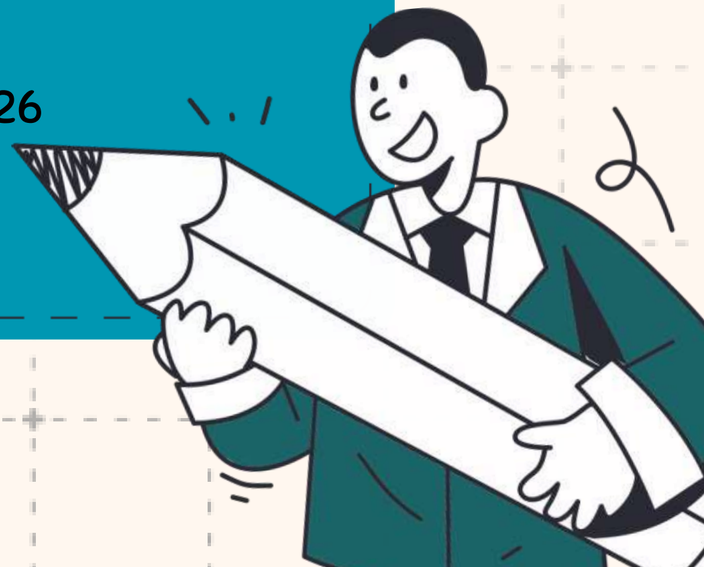
POWERPUFFGALS

NEXUS Writes

YOUR TECH, YOUR STORY

PRESENTED BY: GROUP POWERPUFFGALS
BICOL UNIVERSITY POLANGUI CAMPUS
IT 112: WEB SYSTEMS AND TECHNOLOGIES | A.Y. 2025-2026



PROJECT INTRODUCTION

NEXUS *Writes*

A SPECIALIZED FULL-STACK PUBLISHING PLATFORM FOR THE IT COMMUNITY — NOT A GENERIC BLOG. IT'S WHERE TECHNICAL DOCUMENTATION MEETS PROFESSIONAL NETWORKING.

THE PROBLEM IT SOLVES

Students and tech professionals lack a centralized, structured space to archive laboratory activities, share verified technical expertise, and connect with peers — NEXUSWrites fills that gap.

TECHNOLOGY STACK

Frontend:

- HTML5, CSS3, Bootstrap 5.3
- Vanilla JavaScript (modular JS files per page)
- Font Awesome icons, Google Fonts (Montserrat, Dancing Script, Inter)
- PWA-ready (manifest.json + service worker sw.js for offline caching)

Backend:

- Node.js + Express.js (v5)
- MongoDB + Mongoose ODM (hosted on Atlas, dbName: test)
- JWT (jsonwebtoken) for session authentication
- Bcrypt.js for password hashing

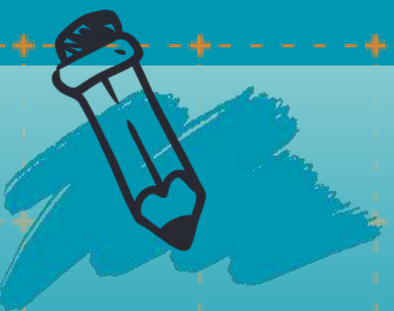
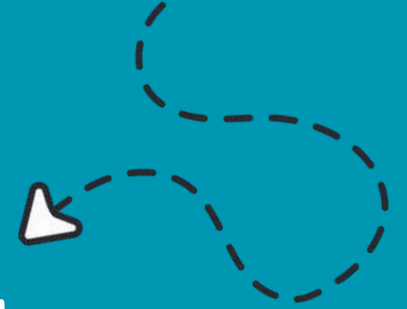
Cloud & Media:

- Cloudinary for image hosting (avatars, post images) – only URLs stored in MongoDB
- Multer + multer-storage-cloudinary for file upload middleware

Deployment:

- Backend hosted on Render (final-project-backend.onrender.com)
- Frontend served statically via Express from the same server



SYSTEM FEATURES OVERVIEW

Feature 1 – Age-Gated Secure Authentication

- Two-page registration form: identity first, then profile setup
- Server-side age calculation – must be 18+ to register (enforced in both authController.js and userController.js)
- Passwords hashed with Bcrypt.js via a pre-save hook in User.js
- Sessions managed with JWT (1-day expiry), stored in localStorage
- Login errors shown via a custom Bootstrap modal, not browser alerts

Feature 2 – Tech-Centric CRUD with Cloud Media

- Dashboard (dashboard.html) lets users create, edit, and delete tutorials
- Post fields: Title, Category (10 predefined + custom), Content, Tags, and an optional Image
- Images uploaded to Cloudinary via Multer middleware; only the URL string is saved in MongoDB
- Posts support Likes (toggle, stored as an array of User ObjectIds in Post.js)
- "Load More" pagination on the feed

Feature 3 – Hierarchical Discussion System (Nested Comments)

- Comments are embedded directly inside the Post document (Post.js uses a recursive commentSchema that adds a replies array to itself)
- Supports unlimited nesting depth – the autoPopulateUser middleware populates user names/avatars up to 10 levels deep
- Users can reply to any comment; the UI threads them visually

Comment/reply actions trigger Notifications to the post or comment author

Feature 4 – Live GitHub Portfolio Integration

- github.html allows any user to search a GitHub username
- Backend calls the GitHub REST API (/users/:username/repos) via Axios, returning the top 5 most recent public repos (name, description, language, stars, URL)
- "Sync to My Profile" saves the GitHub username to the user's MongoDB document via PUT /api/users/update
- The linked repos then display on the public profile.html as a Technical Portfolio section

Feature 5 – Dynamic Technical Categorization & Search

- homesearch.html provides a real-time search with instant suggestions
- Category tabs: General, Web Development, Mobile Apps, AI & Machine Learning, Data Science, UI/UX Design, Cybersecurity, Cloud Computing, Blockchain, Internet of Things
- Backend supports GET /api/posts/search for keyword-based filtering
- Posts can also be filtered by category through tab switching on the search page

Feature 6 – Following System & Personalized Feed

- Users can follow/unfollow others; counts stored in followers[] and following[] arrays in User.js
- following.html fetches all posts and filters to show only those from followed authors
- GET /api/posts/following-feed is a dedicated protected route for this
- Follow actions trigger a Notification to the target user

Feature 7 – Real-Time Notifications

- Notifications generated on: likes, comments, replies, and follows
- Notification.js model stores recipient, sender, type, linked post, and isRead status
- Bell icon in the navbar shows an unread badge count
- PUT /api/notifications/read marks all as read instantly

Feature 8 – Settings & Personalization

- settings.html allows theme (light/dark), font size, language, and timezone preferences
- Saved to the user's settings sub-document in MongoDB via PUT /api/users/update
- Dark mode applied globally via localStorage and CSS class toggling across all pages

CHALLENGES ENCOUNTERED

Recursive Nested Comments

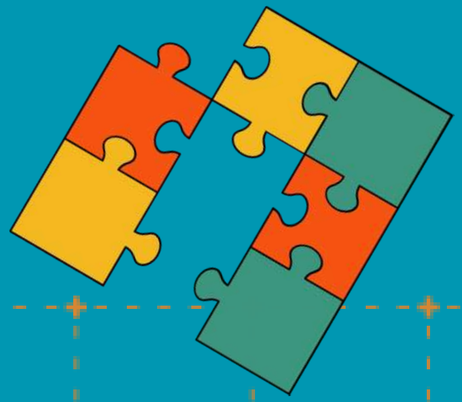
- Problem: Standard flat comment schemas don't support infinite threading

Solution: Made commentSchema reference itself using .add({ replies: [commentSchema] }) and wrote a recursive autoPopulateUser pre-query middleware to populate author names at every depth level

CHALLENGES ENCOUNTERED

JWT Token Inconsistency Across Pages

- Problem: Some pages couldn't find the token because it was saved under different localStorage keys (token vs inside nexusUser object)
- Solution: Standardized token retrieval – login saves to both token key and inside the nexusUser object; all pages pull from nexusUser.token

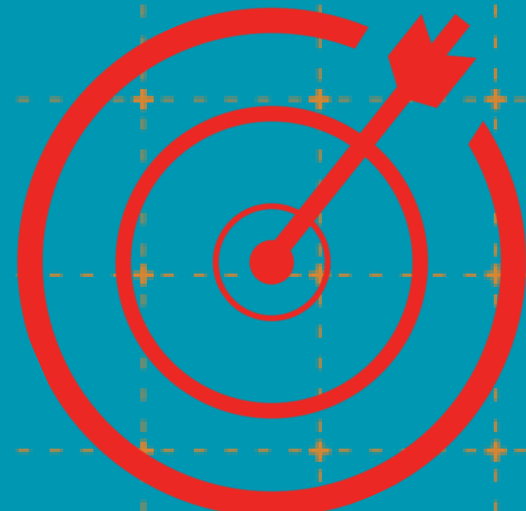


CHALLENGES ENCOUNTERED

Dark Mode Flash on Page Load

- Problem: Pages would briefly show white/light mode before the dark class was applied by JavaScript

Solution: Called syncTheme() at the very top of each JS file before DOMContentLoaded, so the class is applied before the browser paints



CHALLENGES ENCOUNTERED

Cloudinary Image Rendering Performance

- Problem: Storing large base64 images directly in MongoDB caused slowdowns
- Solution: Shifted entirely to Cloudinary – only the URL string is stored, reducing document size and improving load times dramatically

CHALLENGES ENCOUNTERED

GitHub API CORS & Rate Limiting

- Problem: Calling GitHub API directly from the frontend hit CORS issues
- Solution: Proxied all GitHub calls through our own Express backend using Axios, which also adds a proper User-Agent header required by GitHub's API

FUTURE IMPROVEMENTS

- Real-time notifications using WebSockets (Socket.io) instead of polling
- Rich text editor (e.g., TipTap or Quill.js) for code syntax highlighting in posts
- Search by user in addition to posts and categories
- Email verification on signup for stronger account authenticity
- Post bookmarking – save posts for later reading
- Analytics dashboard – show authors how many views and likes their posts get over time
- Mobile app – convert the existing PWA into a full React Native app